

U-Net Image Segmentation on the Semantic Segmentation of Underwater Imagery Dataset

CS 685

Colin Aslett

Introduction

This project implements a U-Net model from scratch to perform image segmentation on the Semantic Segmentation of Underwater Imagery dataset (SUIM). Specifically, the U-Net built for this project will not be using a pre-trained encoder but will still follow the standard encoder-decoder structure of a U-Net. The SUIM dataset contains pixel-level ground truth masks for each image [3]. This allows for the U-Net to be evaluated on its ability to assign every pixel to one of eight classes. The goal of the project is to evaluate the performance of the built from scratch U-Net and compare it to SUIM-Net and other U-Net variations, primarily using mean intersection over union (mIOU).

Dataset

The SUIM dataset contains 1635 images with 1525 for training and 110 for testing [3]. Each image has a corresponding ground truth mask with eight class labels: Background/Waterbody, Human Divers, Aquatic Plants, Wrecks/Ruins, Robots/Instruments, Reefs/Invertebrates, Fish/Vertebrates, and Sea-floor. The dataset is highly class imbalanced, with the Background and Reefs classes representing over 60% of total labelled pixels. Figure 1 shows the class distribution for the eight classes in the SUIM dataset.

Class Label	Class Frequency (Total Labelled Pixels)
Background	31.75%
Human Diver	1.96%
Aquatic Plants	2.19%
Wrecks/Ruins	7.90%
Robots/Instruments	0.35%
Reefs	33.86%
Fish	7.65%
Sea-Floor	14.35%

Figure 1: SUIM class distribution, training and testing

Challenges of SUIM Dataset

A major challenge with the SUIM dataset is that all images are underwater. Underwater imagery often contains blur, low contrast, color distortion, and uneven lighting. This can make two images of the same class appear visually different. Figure 2 shows an example where lighting and contrast can impact the U-Net's ability to predict the wrong class.

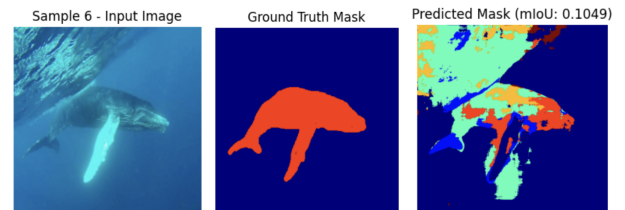


Figure 2: U-Net prediction (far-right) on an example SUIM image (far-left). Ground truth mask in the center

An additional challenge is the small size of the dataset. This can make it significantly harder for a model without a pretrained encoder to perform well. The U-Net was specifically chosen to handle this challenge, with its encoder-decoder structure and skip connections making it strong on pixel-level segmentation when the dataset is small. Class imbalance, which was previously mentioned and shown in Figure 1, is an additional challenge for the model. With some classes making up less than 5% of the dataset, the two main background classes dominate the dataset. To address this, a weighted hybrid loss is used. This is further explained in the Methodology section of this paper.

U-Net

A U-Net was chosen for this project because it is efficient to train and performs strongly on segmentation problems with limited training datasets. Tensorflow was used to implement the U-Net's architecture. The U-Net implemented follows the encoder-decoder structure [2]. The full architecture of the implemented U-Net for this project is shown in Figure 3.

U-Net Encoder

The main purpose of the U-Net encoder is to learn the semantic features of each image. The encoder contains four downsampling stages (convolution block and max pooling layer) followed by a bottleneck. Each image is downsampled from a starting input size of $512 \times 512 \times 3$ down to $32 \times 32 \times 512$. Each convolution block contains two 3×3 convolutional layers with stride 1, same padding, ReLU activation, and batch normalization. The spatial resolution stays the same within each convolution block. Batch normalizations are used after each convolution layer to stabilize training. After each convolution block, a downsampling through max pooling (2×2) is performed with the number of filters doubled. Dropout is added to the deepest three convolution blocks as a regularization tool to prevent overfitting. The last convolution block does not have a downsampling layer after it, instead it acts as the bottleneck. The bottleneck is the point in which the model has the strongest high-level understanding but the smallest spatial resolution.

U-Net Decoder

The U-Net decoder gradually upsamples the image back to its original input image size. The decoder contains four upsampling layers and four decoder convolution blocks. The decoder blocks are the same structure as the encoder convolution blocks but do not contain any dropouts. This U-Net model uses bilinear upsampling followed by a regular convolution, which is done to reduce possible checkerboard artifacts. After each upsampling layer, the decoder feature map is concatenated with the corresponding encoder feature map through a skip connection. These skip connections help the model recover fine details that were potentially lost during downsampling.

U-Net Output

The final layer is a 1×1 convolution with eight filters followed by softmax activation. This produces a $512 \times 512 \times 8$ probability map where each pixel is then assigned a class based on the highest class probability from the probability map.

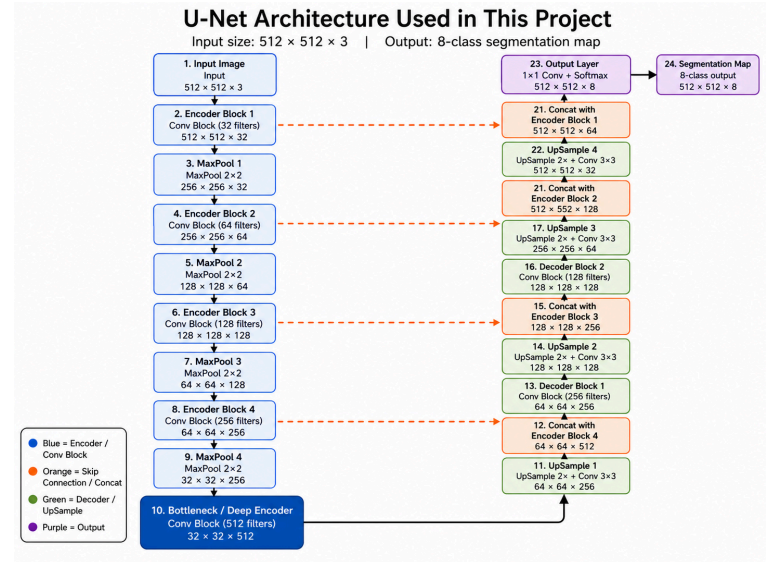


Figure 3: U-Net model architecture for this project

Preprocessing

Excessive preprocessing can reduce a model's ability to generalize by making the image too different from the training data. Given this, the following preprocessing steps were used. First, random scale augmentation was applied to scale the training images and their ground truth masks to between $0.75x$ and $1.25x$. The images and ground truth masks were then resized to fit within 512×512 while preserving their original aspect ratio. Padding was added when needed and these pixels were given ignore labels. The ground truth masks were resized using nearest-neighbor interpolation so class labels were not changed. Before preprocessing, images in the SUIM dataset had a median height and width of 640 and 480 respectively. Second, the ground truth masks were converted from RGB to a single channel class mask where each pixel was assigned a class label from 0 to 7. Third, the pixel intensity values were normalized to be between 0 and 1. Fourth, random geometric and image augmentation was added to select images. Each training image had a 50% chance of having horizontal flip applied. All images had random adjustments to brightness, contrast, hue, and saturation. Clipping was applied to prevent these augmentations from creating invalid pixel values. Validation and test image sets were only resized, padded, normalized, and converted to class label masks.

Methodology: Class Imbalance and Weighted Loss

To handle the class imbalance in the SUIM dataset, a hybrid weighted loss was used. Without weighted loss, the model will predict the dominant classes for the non-dominant classes with little decrease in loss. The weighted loss encourages the model to recognize the non-dominant classes. Although the class distribution, shown in figure 1, shows that some classes are much rarer than other classes, the implemented loss method uses moderated class weights. For example, the model is punished 7.5x more for incorrectly choosing the robot label than choosing reef. The hybrid loss is a combination of weighted sparse categorical cross-entropy loss (WSCCE) and Dice loss. WSCCE improves the pixel classification by increasing the contribution from non-dominant classes. Dice loss encourages smoother region overlap between the predicted and ground truth masks. Pixels with the ignore label were excluded from the loss calculation so padding had no effect on training.

Methodology: Hyperparameter and Training

The U-Net for this project was trained on the following hyperparameters. Epochs were set at 100. Second, the learning rate was 0.0003 with a $\frac{1}{2}$ reduction if the validation mean IOU did not improve over five epochs. The dropout rate for the convolution blocks was 0.2. Batch size was set at eight. The number of starting filters was set at 32. The model was optimized using AdamW with weight decay of 0.0001. The training dataset was split 80/20 for training and validation respectively. The model was trained on Google Colab A100 GPU. Training took roughly 35 minutes and ~30 seconds per epoch. Training and validation plots over the training process for pixel accuracy, hybrid loss, and mean IOU are shown in Figure 4. All three plots show either an increase in pixel accuracy and mean IOU or a decrease in loss over the course of the 100 epochs. The plots do not show severe overfitting since validation metrics improve over 100 epochs. However, there is a sizable gap between the validation and training metrics. This suggests a potential generalization gap. Interestingly, the test metrics, shown in Figure 6, are better than the

validation metrics. It is possible that the validation set that is randomly chosen had images that were of harder difficulty.

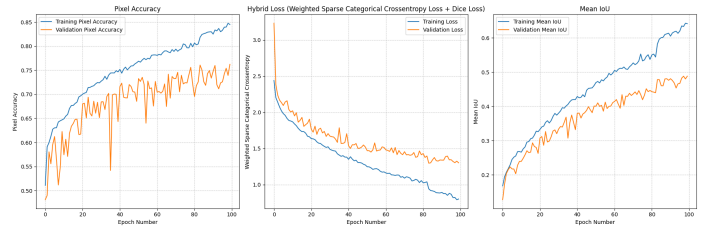
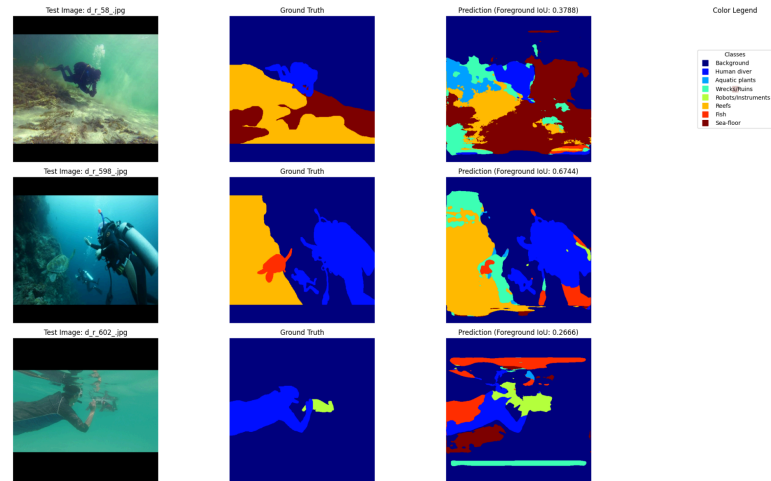


Figure 4: Training and Validation Loss Plots

Visual Results

This section shows the visual prediction of a small subset of test images. Additional visual results are included in the file: Visual_Results_CS_685_CA.pdf. Images shown in Figure 5 show that the model performs strongly on predicting background pixel labels. Foreground objects present a significantly harder challenge for the model, especially if they are small and/or thin. Additionally, despite the geometric augmentations made to improve the model's generalization skills, it still struggles with lighting, contrast, saturation, and hue differences. Both test images d_r_58.png and d_r_602.png show the model predicting water ripples and lighting changes as a separate class. The model generally does strong with identifying human divers in the image, even if it is unable to exactly match the boundaries of the diver.



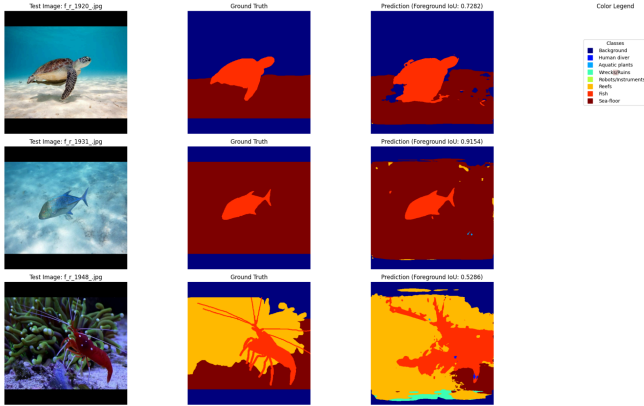


Figure 5: Original test image is far left, ground truth mask is center, U-Net prediction mask is on far right.

Evaluation and Comparison

The results for the model on the training, validation, and test IOU are shown in Figure 6. The model performs the strongest on the most dominant class in the dataset: background. It performs the worst by a large margin on aquatic plants. The model generally does better on the background classes than the foreground classes. This is likely due to the dominance of background pixels in the images. The five foreground classes are: Human Diver (HD), Wrecks/Ruins (WR), Robots (RO), Reefs/Invertebrates (RI), Fish/Vertebrates (FV).

Class Label	Training IOU	Validation IOU	Test IOU
Background	89.27	80.13	87.50
Human Diver	75.64	49.98	63.11
Aquatic Plants	35.91	08.28	17.72
Wrecks/Ruins	74.59	42.32	48.12
Robots & Instruments	76.88	48.51	49.05
Reefs	76.34	68.47	59.46
Fish	69.02	53.78	60.45
Sea-Floor	55.15	38.94	65.50
Mean IOU	69.10	48.80	56.37
Foreground IOU	74.49	52.61	56.04

Figure 6: Training, Validation, and Test Image Data IOU

Figure 7 shows the benchmark results from Table II of the SUIM paper [1]. F-Score is shown in the upper half and mIOU is shown in the lower half. There are several reasons why this project's U-Net performs worse across the board than SUIM-Net and U-NetRGB. First, this project's U-Net does not have a pretrained encoder while SUIM-Net uses a VGG-16 encoder [1]. Second, this project's U-Net is significantly smaller than the U-NetRGB shown in Figure 7. Computation costs limited the options of how extensive the U-Net could be for this project.

Model	HD	WR	RO	RI	FV	Combined	Saliency Pred.
F₁ (→)							
FCN8s _{CNN}	76.34 ± 2.24	70.24 ± 2.26	39.83 ± 3.87	61.66 ± 2.36	76.24 ± 1.87	64.86 ± 2.52	75.62 ± 1.79
FCN8s _{VG}	89.10 ± 1.50	82.03 ± 1.94	74.01 ± 3.23	79.19 ± 2.27	90.46 ± 1.18	82.96 ± 2.02	89.63 ± 1.24
SegNet _{CNN}	59.60 ± 2.02	41.60 ± 1.65	31.77 ± 3.03	41.88 ± 2.66	60.08 ± 1.91	46.97 ± 2.25	56.96 ± 1.58
SegNet _{ResNet}	80.52 ± 3.26	77.65 ± 3.15	62.45 ± 3.90	82.30 ± 1.96	91.47 ± 1.01	76.88 ± 2.66	86.88 ± 1.83
UNet _{GRAY}	85.47 ± 2.21	79.77 ± 2.01	69.95 ± 3.31	69.95 ± 2.57	84.47 ± 1.39	75.12 ± 2.30	83.96 ± 1.40
UNet _{RGB}	89.60 ± 1.84	86.17 ± 1.73	68.87 ± 3.30	79.24 ± 2.70	91.35 ± 1.14	83.05 ± 2.14	89.99 ± 1.29
PSPNet _{MobileNet}	80.21 ± 1.19	70.94 ± 1.61	72.04 ± 2.21	72.65 ± 1.62	79.19 ± 1.74	76.01 ± 1.67	78.42 ± 1.59
DeepLabv3	89.68 ± 2.09	77.73 ± 2.18	72.72 ± 3.35	78.28 ± 2.70	87.95 ± 1.59	81.27 ± 2.30	85.94 ± 1.72
SUIM-Net _{RGB}	89.04 ± 1.31	65.37 ± 2.22	74.18 ± 2.11	71.92 ± 1.80	84.36 ± 1.37	78.86 ± 1.79	81.36 ± 1.72
SUIM-Net _{VG}	93.56 ± 0.98	86.02 ± 1.03	78.06 ± 1.50	83.49 ± 1.39	93.73 ± 0.87	86.97 ± 1.15	91.91 ± 0.85
mIOU (→)							
FCN8s _{CNN}	67.27 ± 2.50	81.64 ± 2.16	36.44 ± 3.67	78.72 ± 2.50	70.25 ± 2.28	66.86 ± 2.62	75.63 ± 1.89
FCN8s _{VG}	79.86 ± 1.50	85.77 ± 2.09	65.05 ± 3.00	85.23 ± 2.07	81.18 ± 1.46	79.42 ± 2.02	85.22 ± 1.24
SegNet _{CNN}	62.76 ± 2.35	66.75 ± 2.57	36.63 ± 3.18	63.46 ± 3.18	62.48 ± 2.32	58.42 ± 2.71	65.90 ± 2.12
SegNet _{ResNet}	74.00 ± 2.88	82.68 ± 2.94	58.63 ± 3.61	89.61 ± 1.15	82.96 ± 1.38	77.58 ± 2.39	83.09 ± 1.96
UNet _{GRAY}	78.33 ± 2.34	85.14 ± 2.14	57.25 ± 3.00	79.96 ± 2.55	78.00 ± 1.90	75.74 ± 2.38	82.77 ± 1.59
UNet _{RGB}	81.17 ± 2.02	87.54 ± 2.00	62.07 ± 3.12	83.69 ± 2.58	83.83 ± 1.47	79.66 ± 2.24	85.85 ± 1.54
PSPNet _{MobileNet}	75.76 ± 1.47	86.82 ± 1.26	72.66 ± 1.47	85.16 ± 1.65	74.67 ± 1.90	77.41 ± 1.56	80.87 ± 1.56
DeepLabv3	80.78 ± 2.07	85.17 ± 2.08	66.03 ± 3.16	83.96 ± 2.52	79.62 ± 1.85	79.10 ± 2.34	83.55 ± 1.65
SUIM-Net _{RGB}	81.12 ± 1.76	80.68 ± 1.74	65.79 ± 2.10	84.90 ± 1.77	76.81 ± 1.82	77.77 ± 1.64	80.86 ± 1.64
SUIM-Net _{VG}	85.09 ± 1.45	89.90 ± 1.29	72.49 ± 1.61	89.51 ± 1.25	83.78 ± 1.55	84.14 ± 1.43	87.67 ± 1.24

Figure 7: Table II from SUIM-Net Paper [1]

Conclusion

This project aimed to show that a built from scratch U-Net could achieve reasonable performance on the SUIM dataset. Additionally, this project showed that a U-Net without a pre-trained encoder could perform well on a small and challenging dataset. Next steps to improve the performance of this model would be to increase data augmentation, add a pre-trained encoder, and/or increase the depth of the U-Net model.

Citations

[1] Islam, M. J., Edge, C., Xiao, Y., Luo, P., Mehtaz, M., Morse, C., Enan, S. S., & Sattar, J. (2020). *Semantic segmentation of underwater imagery: Dataset and benchmark*. 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 1769–1776. <https://doi.org/10.1109/IROS45743.2020.9340821>

[2] Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional networks for biomedical image segmentation*. In N. Navab, J. Hornegger, W. M. Wells, & A. F. Frangi (Eds.), *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015* (pp.

234–241). Springer.

https://doi.org/10.1007/978-3-319-24574-4_28

[3] Minnesota Interactive Robotics and Vision Laboratory. (n.d.). *SUIM dataset*. University of Minnesota. Retrieved May 3, 2026, from <https://irvlab.cs.umn.edu/resources/suim-dataset>